

GOMAL UNIVERSITY DERA ISMAIL KHAN

United College Hazrat Sultan Bahoo Jhang

Department of Computer Science

FINAL YEAR PROJECT DOCUMENTATION

BuildMart

Online Hardware Store – Full-Stack E-Commerce Web Application

Submitted by:

Uswa Babar (Reg: 48-UCHSB-23)

Nimra Jalil (Reg: 55-UCHSB-23)

Rimsha Yaqoob (Reg: 46-UCHSB-23)

Asma Shahamand (Reg: 61-UCHSB-23)

Asma Nawaz (Reg: 69-UCHSB-23)

Supervised by:

Sir Mudassar

Head of Department

Session 2023-2027 | BSCS

Date: March 2026

Declaration

We hereby declare that this project entitled "BuildMart – Online Hardware Store" is our original work, carried out as part of the Final Year Project requirement for the degree of Bachelor of Science in Computer Science (BSCS) at the United College Hazrat Sultan Bahoo Jhang, affiliated with Gomal University Dera Ismail Khan.

We confirm that this project has not been submitted in whole or in part to any other university or institution for the award of any degree or diploma. All references to previously published work by other authors have been duly acknowledged and cited in the bibliography section of this document.

We further declare that the intellectual content of this project is the product of our own collaborative work and that all team members have contributed meaningfully to the research, design, development, testing, and documentation phases of the project.

Team Members:

1. Uswa Babar _____
2. Nimra Jalil _____
3. Rimsha Yaqoob _____
4. Asma Shahamand _____
5. Asma Nawaz _____

Date: March 2026

Place: Jhang, Punjab, Pakistan

Certificate of Approval

This is to certify that the Final Year Project entitled "BuildMart – Online Hardware Store" has been carried out by the following students of BSCS program at United College Hazrat Sultan Bahoo Jhang, affiliated with Gomal University Dera Ismail Khan, under the supervision of Sir Mudassar, Head of Department.

Sr. No	Name	Registration No	Roll No
1	Uswa Babar	48-UCHSB-23	23004
2	Nimra Jalil	55-UCHSB-23	23011
3	Rimsha Yaqoob	46-UCHSB-23	23002
4	Asma Shahamand	61-UCHSB-23	23017
5	Asma Nawaz	69-UCHSB-23	23025

The project has been completed satisfactorily and is approved for submission in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science.

Supervisor:

Sir Mudassar

Head of Department, Computer Science

External Examiner:

Acknowledgements

All praise and gratitude is due to Almighty Allah, the Most Gracious, the Most Merciful, who bestowed upon us the knowledge, wisdom, and strength to undertake and complete this project successfully. Without His blessings and guidance, this achievement would not have been possible.

We would like to express our deepest and most sincere gratitude to our project supervisor, Sir Mudassar, Head of Department of Computer Science at United College Hazrat Sultan Bahoo Jhang. His invaluable guidance, constructive criticism, continuous encouragement, and expert advice throughout the duration of this project have been instrumental in shaping the final outcome. His willingness to dedicate his time and share his knowledge has been a source of great motivation for our entire team.

We extend our heartfelt thanks to the faculty members of the Department of Computer Science for their support, academic guidance, and for creating an environment conducive to learning and innovation. Their teachings over the course of our BSCS program have laid the foundation upon which this project was built.

We are deeply grateful to Gomal University Dera Ismail Khan and United College Hazrat Sultan Bahoo Jhang for providing us with the academic framework, resources, and opportunities that enabled us to pursue our education and this project. The institutional support has been a cornerstone of our academic journey.

We would also like to acknowledge the contributions of our classmates and peers who provided valuable feedback, participated in testing sessions, and offered suggestions that helped improve the quality of the BuildMart system. Their collaborative spirit has been truly appreciated.

Special thanks to the open-source community and the developers behind Node.js, Express.js, MongoDB, and the numerous npm packages that formed the technological backbone of this project. The availability of well-documented, high-quality tools made the development process more efficient and productive.

Last but certainly not least, we owe an immeasurable debt of gratitude to our families. Their unconditional love, constant prayers, financial support, patience during long working hours, and unwavering belief in our abilities have been the driving force behind our perseverance. This project is as much a result of their sacrifices as it is of our efforts. May Allah reward everyone who contributed to the success of this project.

Abstract

BuildMart is a comprehensive full-stack e-commerce web application designed and developed as an Online Hardware Store platform. In the contemporary digital era, the demand for convenient, reliable, and efficient online shopping platforms has increased significantly, driven by the need for streamlined product discovery, secure transaction processing, and accessible delivery services. This project provides a centralized digital platform for managing hardware products, processing customer orders, and handling administrative tasks through an interactive and intuitive web interface.

The system caters to two primary user roles: customers and administrators. Customers can browse and view a comprehensive catalog of hardware products organized by categories including Power Tools, Hand Tools, Plumbing, Electrical, Paint and Supplies, and Building Materials. They can search products by name, add items to their shopping cart with quantity control, proceed through a checkout process with delivery address, phone number, payment method selection, and optional gift messaging. The system also features a Limited Time Deals section with integrated live countdown timers, JWT-based user authentication, and personal order history tracking through a four-stage lifecycle: Pending, Confirmed, Shipped, and Delivered.

On the administrative side, the system provides a secure admin panel protected by two-layer JWT authentication. Administrators can perform complete product management operations including adding, editing, and deleting products with image upload capabilities powered by Multer. They can manage the order workflow by updating order statuses through each stage of the lifecycle, create and manage time-limited promotional deals with images and countdown deadlines, and activate or deactivate promotional offers as needed.

The application architecture follows a three-tier client-server model. The presentation tier is built using HTML5, CSS3, and Vanilla JavaScript for a responsive and accessible user interface. The application tier leverages Node.js with Express.js for server-side business logic, routing, and middleware management. The data tier utilizes MongoDB Atlas as the cloud-hosted NoSQL database for flexible and scalable data storage. Additional technologies employed include bcryptjs for secure password hashing,

express-validator for robust input validation, and JSON Web Tokens for stateless session authentication.

The system was developed following the Agile methodology, allowing for iterative development cycles, continuous feedback incorporation, and adaptive planning. Comprehensive testing was conducted including unit testing, integration testing, performance testing, and security testing to ensure reliability and quality. The fully responsive design ensures a seamless user experience across desktops, laptops, tablets, and smartphones.

Keywords: E-Commerce, Online Hardware Store, Web Application, Node.js, Express.js, MongoDB Atlas, JWT Authentication, Full-Stack Development, Shopping Cart, Order Management, RESTful API

Table of Contents

List of Tables

Table 2.1: Comparison of Existing E-Commerce Hardware Platforms

Table 3.1: Document Conventions and Abbreviations

Table 3.2: Operating Environment Requirements

Table 3.3: Functional Requirements - Customer Module

Table 3.4: Functional Requirements - Admin Module

Table 3.5: Non-Functional Requirements Summary

Table 4.1: Use Case UC-01 - User Registration

Table 4.2: Use Case UC-02 - User Login

Table 4.3: Use Case UC-03 - Browse Products

Table 4.4: Use Case UC-04 - Add to Cart

Table 4.5: Use Case UC-05 - Place Order

Table 4.6: Use Case UC-06 - Manage Products (Admin)

Table 4.7: Use Case UC-07 - Manage Orders (Admin)

Table 4.8: Use Case UC-08 - Manage Deals (Admin)

Table 4.9: Use Case UC-09 - Search Products

Table 4.10: Use Case UC-10 - View Order History

Table 5.1: Database Collections and Key Fields

Table 5.2: Authentication API Endpoints

Table 5.3: Product API Endpoints

Table 5.4: Order API Endpoints

Table 5.5: Cart and Deal API Endpoints

Table 6.1: Core Dependencies and Libraries

Table 6.2: Development Tools

Table 7.1-7.10: Test Cases TC-01 through TC-10

Table 7.11: Testing Summary Results

Table A.1: Product Catalog

Table A.2: Default Admin Credentials

Table A.3: Project Team and Roles

List of Figures

Figure 2.1: Growth of E-Commerce in Pakistan

Figure 4.1: Use Case Diagram - Customer Actor

Figure 4.2: Use Case Diagram - Admin Actor

Figure 4.3: Use Case Diagram - Complete System

Figure 5.1: Three-Tier Architecture Diagram

Figure 5.2: System Architecture Overview

Figure 5.3: Entity Relationship Diagram (ERD)

Figure 5.4: Data Flow Diagram - Level 0

Figure 5.5: Data Flow Diagram - Level 1

Figure 5.6: DFD Level 2 - Authentication Module

Figure 5.7: Sequence Diagram - User Registration

Figure 5.8: Sequence Diagram - Add to Cart and Checkout

Figure 5.9: Sequence Diagram - Order Lifecycle

Figure 5.10: Activity Diagram - Order Process

Figure 5.11: Class Diagram - System Models

Figure 5.12: Component Diagram

Figure 5.13: Deployment Diagram

Figure 6.1: Project Directory Structure

Figure 6.2: Frontend Page Flow

Figure 6.3: Backend API Architecture

Figure 6.4-6.10: Application Screenshots

Figure 7.1: Test Results Summary Chart

Figure 7.2: Performance Test Results



Chapter 1: Introduction

1.1 Introduction

In the rapidly evolving landscape of digital commerce, the hardware and construction supply industry is undergoing a profound transformation. Traditional brick-and-mortar hardware stores, while still vital to local communities, face increasing pressure to establish digital presences that complement their physical operations. The global shift toward e-commerce, accelerated by changing consumer preferences and technological advancements, has created both an opportunity and a necessity for hardware retailers to offer online shopping experiences that match the convenience, reliability, and breadth of selection that modern consumers expect.

BuildMart is a comprehensive full-stack e-commerce web application designed and developed specifically for the online hardware retail sector. The system aims to address the multifaceted challenges associated with selling hardware products, tools, and building materials through a digital platform, from product cataloging and inventory management through customer order processing and delivery tracking. By providing a centralized web-based platform, BuildMart enables hardware store owners and customers to interact seamlessly, maintaining clear communication channels and ensuring that the entire purchase lifecycle is conducted efficiently, securely, and transparently.

The motivation behind the development of BuildMart stems from a thorough analysis of the challenges faced by hardware retailers in Pakistan's growing but underserved e-commerce market. These challenges include the difficulty of presenting diverse product categories effectively online, the complexity of managing inventory across multiple product types, the need for secure and flexible payment options suited to the Pakistani market, and the importance of providing reliable order tracking for products that are often heavy, bulky, or fragile. BuildMart addresses each of these challenges through a carefully designed system architecture that leverages modern web technologies to deliver a robust, scalable, and user-friendly solution.

The system is built upon a three-tier client-server architecture that separates the presentation, application, and data layers. The frontend is developed using HTML5, CSS3, and Vanilla JavaScript, providing a responsive and accessible user interface that works seamlessly across all devices. The backend is powered by Node.js with Express.js, offering a performant and flexible server-side framework for handling business logic, authentication, and API management. MongoDB Atlas serves as the cloud-hosted NoSQL database, providing flexible schema design, horizontal scalability, and built-in high availability.

BuildMart supports two primary user roles: customers and administrators. Customers can browse categorized hardware products, search by name, manage their shopping cart, place orders with full delivery details, and track order status through a defined lifecycle. Administrators have access to a secure dashboard where they can manage products, process orders, configure promotional deals, and monitor system activity. The system incorporates JWT-based authentication, bcryptjs password hashing, and two-layer admin protection to ensure security at every level.

1.2 Background and Context

The e-commerce sector in Pakistan has experienced remarkable growth over the past decade, driven by increasing internet penetration, the proliferation of smartphones, improved digital payment infrastructure, and a young, tech-savvy population. According to the Pakistan Bureau of Statistics and various industry reports, the country's e-commerce market has been growing at a compound annual rate exceeding 25%, with projections indicating continued acceleration in the coming years.

Despite this growth, the hardware and construction supply segment remains significantly underrepresented in the Pakistani e-commerce landscape. While platforms like Daraz and other general marketplaces offer some hardware products, dedicated online hardware stores that provide specialized product categorization, bulk ordering capabilities, and contractor-focused features are largely absent. This gap presents a significant opportunity for purpose-built solutions that cater specifically to the unique requirements of hardware retail.

The hardware industry presents distinct challenges for e-commerce implementation. Products range from small, lightweight items like screws and LED bulbs to heavy, bulky materials like cement bags and steel rebar bundles. Product specifications are critical for customers making purchasing decisions, requiring detailed descriptions, accurate categorization, and clear pricing. Many customers in this segment are contractors and professionals who require bulk ordering capabilities and efficient order management.

The development of BuildMart also reflects the broader trend toward data-driven retail management. By digitizing the product catalog, sales process, and order management workflow, hardware store owners can collect and analyze data on product popularity, customer purchasing patterns, seasonal trends, and operational efficiency. This data can inform inventory decisions, marketing strategies, and business growth planning.

1.3 Problem Statement

Hardware retailers in Pakistan face significant challenges in establishing effective online sales channels. The current landscape presents several critical problems that affect both store owners and customers. Customers often find it difficult to locate specific hardware products online, as general marketplaces lack the specialized categorization and product knowledge that hardware shopping requires. The absence of dedicated hardware e-commerce platforms means customers must either visit physical stores or navigate poorly organized product listings on general platforms.

Store owners struggle to manage their product catalogs, track orders, and coordinate deliveries efficiently using manual or semi-manual processes. The lack of a centralized system means that inventory information is often outdated, orders may be lost or delayed, and customers receive inconsistent service. The absence of promotional tools such as time-limited deals and discount management features limits store owners' ability to drive sales and compete effectively.

Security and data integrity present additional concerns. Manual order tracking is prone to errors and does not provide the transparency that customers expect. Even basic digital solutions lack the robust access control and audit mechanisms needed to ensure the confidentiality of customer data and the integrity of financial transactions. There is a

clear need for a purpose-built e-commerce system that addresses these challenges comprehensively while remaining accessible and affordable for small to medium hardware retailers.

1.4 Motivations and Challenges

The motivation for designing the BuildMart online hardware store arose from the growing need for convenient, reliable, and secure digital shopping platforms in the hardware industry. Physical hardware stores are limited by geography and operating hours, while an online platform can serve customers 24/7 from any location. The following key motivations guided the development:

First, providing customers with a seamless and intuitive browsing experience for hardware products organized by meaningful categories such as Power Tools, Hand Tools, Plumbing, Electrical, Paint and Supplies, and Building Materials. Second, implementing secure authentication and payment method selection appropriate for the Pakistani market, including Cash on Delivery and Bank Transfer options. Third, building a fully responsive design that provides an optimal experience on mobile devices, recognizing that a significant portion of Pakistani internet users access the web primarily through smartphones.

Fourth, creating a maintainable and scalable application architecture using modern web technologies that can grow with the business. Fifth, enabling store administrators to manage products, orders, and promotions efficiently through an intuitive dashboard without requiring technical expertise.

The project also presented several significant technical challenges. Implementing a full-stack JavaScript application required proficiency in both frontend DOM manipulation and backend API development. Designing a MongoDB schema that efficiently handles diverse product categories, shopping cart operations, and order lifecycle management involved careful consideration of normalization and denormalization trade-offs. Ensuring consistent responsive layouts across different devices and browsers demanded extensive testing and CSS optimization.

1.5 Goals and Objectives

The primary goals and objectives of the BuildMart system are:

To provide a user-friendly web platform that enables customers to browse, search, and purchase hardware products organized by categories with intuitive navigation, efficient search functionality, and clear presentation of product information including prices in Pakistani Rupees (PKR).

To implement secure user registration and login functionality using JWT-based authentication with bcryptjs password hashing, token-based session management, and role-based access control differentiating between customer and administrator privileges.

To enable administrators to manage the product catalog, order workflows, and promotional deals efficiently through a dedicated admin dashboard with comprehensive CRUD tools, image upload capabilities, and order status management.

To implement a shopping cart system with quantity control, real-time total calculation, and a comprehensive checkout process supporting delivery address, phone number, payment method selection, and optional gift messaging.

To provide order tracking capabilities through a defined lifecycle with four stages: Pending, Confirmed, Shipped, and Delivered, with both customers and administrators able to view and track order progress.

To offer Limited Time Deals with integrated live countdown timers that create urgency and drive sales, with administrative controls for creating, activating, deactivating, and deleting promotional offers.

To ensure data integrity, security, and reliability throughout the system through comprehensive input validation, error handling, secure database connections, and thorough testing.

To deliver a fully responsive design that provides an optimal shopping experience across all devices including desktops, laptops, tablets, and smartphones.

1.6 Scope of the Project

The scope of BuildMart encompasses the design, development, and testing of a complete e-commerce web platform for online hardware retail. The system covers customer-facing features including product browsing and search, shopping cart management, checkout with delivery details, order status tracking, personal order history, and interaction with limited-time deals. Admin-facing features include secure dashboard access, product CRUD operations with image upload, order status management, deal configuration, and system monitoring.

The technical scope includes full-stack development using HTML5, CSS3, JavaScript, Node.js, Express.js, and MongoDB Atlas. The system implements RESTful API design, JWT authentication, file upload handling, input validation, and responsive web design. The project scope excludes real-time payment gateway integration (simulated through payment method selection), integration with external inventory management systems, native mobile application development, and multi-vendor marketplace functionality. These are identified as future enhancements.

1.7 Literature Review

1.7.1 E-Commerce in the Hardware Industry

The evolution of e-commerce has transformed retail across all sectors, with the hardware and construction supply industry increasingly embracing digital channels. Research by Chen et al. (2019) highlights how small businesses can effectively leverage e-commerce platforms to expand their market reach beyond geographical limitations. For hardware retailers, online platforms offer the ability to showcase extensive product catalogs, provide detailed specifications, and offer competitive pricing transparency that physical stores struggle to match.

Kim et al. (2019) emphasize the critical importance of user-friendly website design in e-commerce, finding that effective design elements including simple navigation, high-quality product images, clear pricing information, and secure payment processing significantly improve customer retention and conversion rates. These findings directly informed the design of BuildMart, particularly the emphasis on category-based navigation, responsive product cards, and clear pricing display in PKR.

1.7.2 Authentication and Security in E-Commerce

Security is paramount in e-commerce applications that handle customer personal information and financial transactions. JSON Web Tokens (JWT) have emerged as a popular authentication mechanism for modern web applications due to their stateless nature, scalability, and flexibility. The use of bcrypt-based password hashing provides strong protection for stored credentials by incorporating a configurable work factor that makes brute-force attacks computationally expensive.

1.7.3 NoSQL Databases for E-Commerce

MongoDB's document-oriented model is particularly well-suited for e-commerce applications because it can accommodate diverse product attributes across different categories without requiring rigid schema definitions. A power drill has different attributes than a paint can, and MongoDB's flexible document model handles this diversity naturally. MongoDB Atlas provides managed cloud database services with automated backups, built-in high availability, and secure TLS/SSL connections.

1.7.4 Node.js and Express.js for Server-Side Development

Node.js has established itself as a leading platform for server-side development, with its event-driven, non-blocking I/O model making it particularly suited for handling concurrent connections in web applications. Express.js provides a minimal yet robust framework for building RESTful APIs. The combination enables rapid development of the communication layer between frontend and backend components.

1.7.5 Responsive Design for E-Commerce

With mobile commerce accounting for an increasing share of online transactions globally, responsive design is essential for e-commerce success. Research by Lee et al. (2020) found that mobile-friendly design significantly impacts user satisfaction, cart abandonment rates, and conversion rates. BuildMart's use of CSS3 media queries and flexible layouts ensures optimal shopping experience across all device types.

1.7.6 Shopping Cart and Checkout Optimization

The shopping cart is one of the most critical components of any e-commerce system, serving as the bridge between product browsing and order completion. Research

consistently identifies cart abandonment as a major challenge for online retailers, with average abandonment rates ranging from 60% to 80% across different industries and regions. Studies by Wang et al. (2018) found that the primary reasons for cart abandonment include unexpected costs revealed during checkout, complex or lengthy checkout processes, mandatory account creation requirements, concerns about payment security, and lack of preferred payment methods.

For the Pakistani market specifically, Cash on Delivery (COD) remains the dominant payment method, accounting for approximately 65% of all e-commerce transactions. This preference reflects the relatively low penetration of credit cards and digital wallets in rural and semi-urban areas, as well as a cultural preference for verifying products before completing payment. BuildMart addresses this market reality by offering COD as the default payment option while also supporting Bank Transfer for customers who prefer digital payment methods.

The checkout flow in BuildMart was designed to minimize friction and reduce abandonment. The single-page checkout combines delivery address, phone number, payment selection, and optional gift messaging into a streamlined form that can be completed in under two minutes. Real-time form validation provides immediate feedback on input errors, preventing frustrating form resubmission. The cart summary remains visible throughout the checkout process, giving customers confidence in their order before final submission.

1.7.7 Promotional Strategies in E-Commerce

Limited-time promotional offers have been shown to be highly effective in driving immediate sales action in e-commerce environments. The psychological principle of scarcity, combined with visual urgency cues such as countdown timers, creates a compelling motivation for customers to complete purchases rather than deferring them. Research by Singh et al. (2020) found that promotional strategies including time-limited discounts, flash sales, and seasonal offers significantly increase both traffic and conversion rates for online retailers.

BuildMart implements a sophisticated deal management system that allows administrators to create promotional offers with images, descriptions, discount percentages, and precise countdown deadlines. The live countdown timers on the customer-facing interface update every second using JavaScript `setInterval`, creating a sense of real-time urgency. Administrators can toggle deals between active and inactive states, allowing for flexible promotional scheduling without deleting offer configurations.

1.7.8 The Pakistani E-Commerce Landscape

Pakistan's e-commerce ecosystem has evolved significantly in recent years, driven by several key factors. The country's population of over 220 million, with a median age under 23 years, represents a massive potential customer base. Internet penetration has grown rapidly, reaching approximately 55% of the population, with mobile internet accounting for the vast majority of connections. The launch of 4G/LTE services and declining data costs have made mobile internet access increasingly affordable.

The hardware retail sector in Pakistan is estimated to be worth several billion dollars annually, yet online penetration remains minimal. Most hardware purchasing is conducted through traditional bazaars and wholesale markets, where prices are negotiable and product quality varies significantly. An online platform like BuildMart offers the potential to standardize pricing, provide product quality assurance, enable price comparison across categories, and reach customers in areas with limited physical store access.

Government initiatives to promote digital commerce, including the Digital Pakistan vision and e-commerce policy framework, have created a supportive regulatory environment. The growth of mobile wallets (JazzCash, EasyPaisa) and improved logistics infrastructure are removing traditional barriers to e-commerce adoption. BuildMart is positioned to capitalize on these trends by providing a specialized, user-friendly platform for hardware retail.

1.8 Gap Analysis

After analyzing existing online hardware shopping platforms in Pakistan and internationally, several significant gaps were identified:

Most existing general e-commerce platforms lack specialized category navigation and product filtering for hardware items. Items are often poorly categorized, making it difficult for customers to find specific types of products like plumbing fittings or electrical components.

Limited-time promotional deal management with visual countdown timers is rarely implemented in hardware e-commerce, despite being common in fashion and electronics platforms. Comprehensive order status tracking from placement through delivery with four distinct stages is often incomplete. Administrative dashboards frequently lack robust two-layer security protection. Mobile shopping experiences are often compromised, with cart functionality and checkout processes performing poorly on smaller screens. AI-powered search widgets providing immediate customer support are absent from most hardware platforms.

1.9 Proposed Solution

BuildMart addresses the identified gaps by providing a comprehensive, modern, and secure e-commerce platform specifically designed for online hardware retail. The solution features JWT-secured authentication with role-based access control, a complete order lifecycle with four stages, an integrated search widget, limited-time deals with live countdown timers, bulk ordering support, a fully responsive design, and a secure admin dashboard with two-layer JWT protection. The system uses Node.js with Express.js for the backend, MongoDB Atlas for cloud-based data storage, and HTML5/CSS3/JavaScript for the frontend.

1.10 Project Plan

1.10.1 Work Breakdown Structure

The project was decomposed into the following work packages: Requirement Gathering and Analysis, UI/UX Design and Prototyping, Database Design and Implementation, Backend API Development, Frontend Development, Admin Panel Development, Integration and System Testing, and Documentation and Deployment.

1.10.2 Roles and Responsibility Matrix

WBS	Deliverable	Activity	Duration	Responsible
01	Project Proposal	Planning & Research	16 days	All Members
02	Documentation	SRS Document	8 days	Uswa Babar
03	System Analysis	Market Analysis & Research	5 days	Nimra Jalil
04	System Design	UML Diagrams & DB Design	7 days	Rimsha Yaqoob
05	UI Design	Frontend HTML/CSS/JS	12 days	Asma Shahamand
06	Frontend Coding	HTML5, CSS3, JavaScript	20 days	Asma Nawaz
07	Backend Coding	Node.js, Express, MongoDB	40 days	Uswa Babar
08	Testing	All testing phases	7 days	All Members
09	Maintenance	Bug fixing & optimization	8 days	All Members
10	Final Documentation	Complete project report	15 days	All Members

1.10.3 Project Timeline

The project timeline spanned approximately twelve weeks (three months), from January 2026 to March 2026. Phase 1 covered foundation and authentication (Weeks 1-2). Phase 2 focused on inventory and admin CMS (Weeks 3-4). Phase 3 developed core customer flows (Weeks 5-7). Phase 4 implemented cart and checkout (Weeks 8-9). Phase 5 added marketing features and UI polish (Week 10). Phase 6 built order lifecycle management (Week 11). Phase 7 conducted final testing, bug fixing, and deployment (Week 12).

1.11 Structure of the Document

This documentation is organized into nine chapters. Chapter 1 provides the introduction, background, problem statement, objectives, literature review, and project plan. Chapter 2 presents the software requirement specifications. Chapter 3 contains the system analysis with use case definitions. Chapter 4 describes the system design including architecture, database design, and API design. Chapter 5 details the implementation.

Chapter 6 covers the user interface design. Chapter 7 addresses testing and quality assurance. Chapter 8 presents results and discussion. Chapter 9 concludes with summary and future enhancements. Appendices provide supplementary material.

Chapter 2: Software Requirement Specifications

2.1 Introduction

This chapter presents the comprehensive Software Requirement Specifications (SRS) for the BuildMart Online Hardware Store. The SRS document serves as the definitive reference for all functional and non-functional requirements, providing a detailed description of the system's intended behavior, constraints, and quality attributes. Requirements were gathered through stakeholder interviews, analysis of existing e-commerce platforms, review of hardware retail literature, and iterative refinement through Agile development cycles.

2.2 Purpose

The purpose of this SRS is to comprehensively delineate all features, constraints, and quality requirements of the BuildMart system. It provides a complete specification that enables the development team to design, implement, and test the system systematically. It also serves as a validation tool for stakeholders to verify that the delivered system meets their expectations.

2.3 Document Conventions

Abbreviation	Full Form	Description
ERD	Entity Relationship Diagram	Visual representation of database relationships
DB	Database	Organized collection of structured data
SRS	Software Requirement Specification	Document describing system requirements
UML	Unified Modeling Language	Standard notation for system modeling
DFD	Data Flow Diagram	Diagram showing data flow through the system
API	Application Programming Interface	Interface for software communication
JWT	JSON Web Token	Compact token format for authentication
CRUD	Create, Read, Update, Delete	Basic data manipulation operations
REST	Representational State	Architectural style for web services

Abbreviation	Full Form	Description
	Transfer	
GUI	Graphical User Interface	Visual interface for user interaction
HTTP	Hypertext Transfer Protocol	Protocol for web communication
CSS	Cascading Style Sheets	Styling language for web pages
NoSQL	Not Only SQL	Non-relational database approach
PKR	Pakistani Rupee	Currency unit used in the system
COD	Cash on Delivery	Payment method
TLS	Transport Layer Security	Cryptographic protocol for secure communication

2.4 Project Scope

BuildMart provides a web-based e-commerce platform for online hardware retail. The scope encompasses customer-side product browsing, searching, and cart management; checkout with delivery details and payment method selection; order status tracking through the complete lifecycle; administrator-side product management with CRUD operations and image uploads; order workflow management; promotional deal configuration; and system security with JWT-based authentication. The system is designed for deployment by small to medium hardware retailers serving the Pakistani market.

2.5 Overall Description

2.5.1 System Perspective

BuildMart is a standalone web application operating on a client-server architecture. The frontend runs in the user's web browser and communicates with the backend server through RESTful API endpoints. The backend manages business logic, authentication, and database operations, while MongoDB Atlas provides persistent cloud-hosted data storage.

2.5.2 User Classes and Characteristics

The system supports three distinct user classes. Customers are the primary users who browse products, manage their shopping carts, place orders, and track delivery status. They are expected to have basic computer literacy and familiarity with web browsing. Administrators are privileged users responsible for managing the store's product catalog, processing orders, and configuring promotional deals. Visitors are unauthenticated users who can browse the product catalog and view limited-time deals but must register to make purchases.

2.5.3 Operating Environment

Component	Minimum Requirement	Recommended
Processor	Dual Core	Quad Core or higher
RAM	4 GB	8 GB or more
Storage	20 GB free space	50 GB free space
Operating System	Windows 10 / Ubuntu 20.04 / macOS 10.15	Latest stable version
Web Browser	Chrome 90+ / Firefox 88+ / Edge 90+	Latest stable version
Internet	Broadband connection	High-speed broadband
Server Runtime	Node.js v18+	Node.js v20 LTS
Database	MongoDB Atlas Free Tier	MongoDB Atlas M10+

2.6 Functional Requirements

2.6.1 Customer Module Requirements

Req ID	Requirement	Priority	Description
FR-C01	User Registration	High	Customers create accounts with name, email, and password
FR-C02	User Login	High	Authenticated access with JWT token generation
FR-C03	Browse Products	High	View all products with images, categories, and prices (PKR)
FR-C04	Search Products	High	Search by product name with real-time filtering
FR-C05	Add to Cart	High	Add products to cart with quantity control

Req ID	Requirement	Priority	Description
FR-C06	Checkout	High	Complete order with delivery address, phone, payment method
FR-C07	Order Tracking	High	View order status through lifecycle (Pending to Delivered)
FR-C08	Order History	Medium	Access personal order records
FR-C09	View Deals	Medium	Browse limited-time deals with countdown timers
FR-C10	Product Search	Medium	Interactive search widget for product navigation

2.6.2 Administrator Module Requirements

Req ID	Requirement	Priority	Description
FR-A01	Admin Login	High	Two-layer JWT protected admin authentication
FR-A02	Add Products	High	Create new products with images and price in PKR
FR-A03	Edit Products	High	Modify existing product details and images
FR-A04	Delete Products	High	Remove products from the catalog
FR-A05	View Orders	High	List all customer orders with details
FR-A06	Update Order Status	High	Progress orders through Pending/Confirmed/Shipped/Delivered
FR-A07	Create Deals	Medium	Add promotional deals with images and countdown deadlines
FR-A08	Toggle Deals	Medium	Activate or deactivate promotional offers
FR-A09	Delete Deals	Medium	Remove promotional deals from the system
FR-A10	Dashboard Overview	Low	View summary statistics and system metrics

2.7 Non-Functional Requirements

2.7.1 Performance Requirements

The system shall respond to user requests within 3 seconds under normal conditions. API endpoints shall return responses within 500 milliseconds for simple queries. The system shall support a minimum of 50 concurrent users. Page load times shall not

exceed 5 seconds on a standard broadband connection. MongoDB Atlas shall be configured with appropriate indexes on frequently queried fields.

2.7.2 Security Requirements

All user passwords shall be hashed using bcryptjs with a minimum salt round factor of 10. Authentication shall be implemented using JWT tokens with configurable expiration. Admin routes shall be protected by two-layer JWT verification. All API endpoints shall implement server-side validation using express-validator. Database connections shall use TLS/SSL encryption. Sensitive configuration values shall be stored in environment variables.

2.7.3 Reliability Requirements

The system shall maintain data integrity through client-side and server-side validation. All database operations shall use proper error handling with try-catch blocks. MongoDB Atlas provides automatic failover and data replication. The system shall implement graceful error handling that provides meaningful messages without exposing sensitive technical details.

2.7.4 Usability Requirements

The user interface shall be intuitive and require no formal training. The system shall provide consistent visual design with a professional dark-themed navigation bar. All interactive elements shall provide visual feedback. The responsive design shall ensure full functionality on devices with screen widths from 320 pixels to 2560 pixels.

2.7.5 Scalability and Maintainability

The architecture shall support horizontal scaling through load balancing. MongoDB Atlas shall support scaling through tier upgrades and sharding. The RESTful API shall be stateless to facilitate load distribution. The modular code structure shall allow individual modules to be updated independently. Consistent naming conventions and comprehensive documentation shall facilitate maintenance.

2.8 External Interface Requirements

User Interfaces: Web-based GUI with professional dark-themed navigation bar, responsive layouts, and page loader animations. Hardware Interfaces: Server capable of running Node.js v18+ with minimum 4GB RAM; clients with modern web browsers. Software Interfaces: Node.js with Express.js, MongoDB Atlas via Mongoose, JWT for auth, bcryptjs for hashing, Multer for uploads, express-validator for validation. Communication Interfaces: HTTP/HTTPS with JSON data exchange, TLS/SSL encrypted MongoDB connections.

Chapter 3: System Analysis and Use Cases

3.1 Introduction

This chapter presents a comprehensive use case analysis for the BuildMart system, defining the interactions between system actors and the application's functional components. Ten primary use cases collectively cover all major functional requirements. Each use case is documented with identifier, name, summary, priority, preconditions, postconditions, actors, triggers, main scenario, alternative scenarios, and exception handling.

3.2 Actor Identification

The BuildMart system involves three primary actors: Customers who browse products, manage carts, and place orders; Administrators who manage products, process orders, and configure deals; and the System which performs automated functions such as token validation, data persistence, and countdown timer management. Secondary actors include MongoDB Atlas Database and the File System for uploaded images.

3.3 Use Case UC-01: User Registration

Use Case ID	UC-01
Use Case Name	User Registration
Summary	A new customer creates an account on BuildMart by providing personal information and credentials.
Priority	High
Preconditions	Valid email address and active internet connection. No existing account with the same email.
Post-conditions	New user account created in MongoDB Users collection. JWT token generated and returned. User redirected to home page.
Primary Actor	Customer
Secondary Actor	MongoDB Database, Authentication System
Trigger	User clicks Sign Up button.
Main Scenario	1. User navigates to signup page. 2. Enters name, email, password. 3. Client validates input. 4. POST /api/auth/register. 5. Server validates with express-validator. 6. Checks for duplicate email. 7. Hashes password with bcryptjs. 8. Creates user in

	MongoDB. 9. Generates JWT. 10. Redirects to home.
Alternative Scenario	Email exists: error message suggesting login instead.
Exception Handling	Network errors: connection failure message. Validation errors: field-level messages. Database errors: generic error response.

3.4 Use Case UC-02: User Login

Use Case ID	UC-02
Use Case Name	User Login
Summary	Registered user authenticates with email and password.
Priority	High
Preconditions	Registered account exists in database.
Post-conditions	User authenticated, JWT issued and stored in browser, redirected to home with role-based navigation.
Primary Actor	Customer, Administrator
Secondary Actor	MongoDB Database, JWT Authentication System
Trigger	User clicks Login button.
Main Scenario	1. Navigate to login. 2. Enter email/password. 3. POST /api/auth/login. 4. Server retrieves user by email. 5. Compares password with bcryptjs. 6. Generates JWT with user ID and role. 7. Token stored in localStorage. 8. Redirect with role-based navigation.
Alternative Scenario	Invalid credentials: generic error message to prevent enumeration.
Exception Handling	Network errors: appropriate messages. Token generation failures: server error responses.

3.5 Use Case UC-03: Browse and Search Products

Use Case ID	UC-03
Use Case Name	Browse Hardware Products
Summary	Users browse the product catalog with category filtering and search.
Priority	High
Preconditions	None (public access). Products must exist in database.

Post-conditions	Products displayed in responsive grid with images, names, categories, prices (PKR), and action buttons.
Primary Actor	Customer, Visitor
Secondary Actor	Products API, MongoDB Database
Trigger	Navigate to Shop page or click a category filter.
Main Scenario	1. User clicks Shop. 2. GET /api/products. 3. Server retrieves products from MongoDB. 4. JSON array returned. 5. Frontend renders responsive card grid. 6. Each card shows image, name, category, price (PKR), stock status, Add to Cart button. 7. Filter by category or search by keyword.
Alternative Scenario	No matching products: 'No products found' message with suggestion.
Exception Handling	API errors: fallback message. Slow loading: page loader animation.

3.6 Use Case UC-04: Add to Cart

Use Case ID	UC-04
Use Case Name	Add to Shopping Cart
Summary	Authenticated customer adds a hardware product to their cart.
Priority	High
Preconditions	User must be authenticated with valid JWT token.
Post-conditions	Product added to user's cart in MongoDB Cart collection with specified quantity.
Primary Actor	Customer
Secondary Actor	Cart API, MongoDB Database
Trigger	User clicks Add to Cart button on a product card.
Main Scenario	1. User browses products. 2. Clicks Add to Cart. 3. System verifies JWT. 4. POST /api/cart/add with product ID. 5. Checks if product exists in cart. 6. If new, adds with quantity 1. If existing, increments. 7. Updated cart returned. 8. UI updates.
Alternative Scenario	Not authenticated: redirect to login with message.
Exception Handling	Invalid product IDs: 404 error. Database errors: rollback and error display.

3.7 Use Case UC-05: Place Order (Checkout)

Use Case ID	UC-05
Use Case Name	Place Order
Summary	Customer completes checkout and places an order.
Priority	High
Preconditions	User authenticated. Cart has at least one item.
Post-conditions	New order created with Pending status. Cart cleared. Order appears in history.
Primary Actor	Customer
Secondary Actor	Orders API, Cart API, MongoDB Database
Trigger	User clicks Place Order on checkout page.
Main Scenario	1. Navigate to checkout. 2. System displays cart contents with totals. 3. Enter delivery address. 4. Enter phone number. 5. Select payment method (COD/Bank Transfer). 6. Optionally add gift message. 7. Click Place Order. 8. POST /api/orders. 9. Order created with Pending status. 10. Cart cleared. 11. Confirmation displayed.
Alternative Scenario	Empty cart: button disabled, message to add products first.
Exception Handling	Validation errors for required fields. Network errors allow retry.

3.8 Use Case UC-06: Manage Products (Admin)

Use Case ID	UC-06
Use Case Name	Manage Products
Summary	Admin performs CRUD operations on the product catalog.
Priority	High
Preconditions	Authenticated as admin with valid admin JWT.
Post-conditions	Product catalog updated in MongoDB. Changes reflected in customer product listings.
Primary Actor	Administrator
Secondary Actor	Products API, Multer File Handler, MongoDB Database
Trigger	Admin accesses Products section of Admin Dashboard.
Main Scenario	1. Admin logs in, accesses dashboard. 2. Navigates to Products tab. 3. Add: fills form (name, price PKR, category, description, image upload). 4. POST /api/products with multipart data. 5. Multer processes image. 6. Product created in MongoDB. 7.

	Edit: modifies fields, PUT request. 8. Delete: confirms, DELETE request.
Alternative Scenario	File upload fails: product created without image, admin notified.
Exception Handling	Unauthorized: 403. Invalid file types rejected. Database errors: appropriate responses.

3.9 Use Case UC-07: Manage Orders (Admin)

Use Case ID	UC-07
Use Case Name	Manage Order Workflow
Summary	Admin views and updates order statuses through the lifecycle.
Priority	High
Preconditions	Authenticated as admin. Orders exist in system.
Post-conditions	Order status updated. Changes reflected in customer's order history.
Primary Actor	Administrator
Secondary Actor	Orders API, MongoDB Database
Trigger	Admin accesses Orders section of Admin Dashboard.
Main Scenario	1. Navigate to Orders tab. 2. GET /api/orders. 3. All orders displayed with customer details, items, total (PKR), status. 4. Admin reviews order. 5. PATCH /api/orders/:id/status. 6. Status progresses: Pending -> Confirmed -> Shipped -> Delivered. 7. Updated in both admin and customer views.
Alternative Scenario	Delivered orders: status button disabled.
Exception Handling	Invalid transitions rejected. Concurrent conflicts handled.

3.10 Use Case UC-08: Manage Promotional Deals

Use Case ID	UC-08
Use Case Name	Manage Limited Time Deals
Summary	Admin creates and manages promotional deals with countdowns.
Priority	Medium
Preconditions	Authenticated as admin.
Post-conditions	Deals created, updated, or removed. Active deals displayed on

	customer interface.
Primary Actor	Administrator
Secondary Actor	Deals API, MongoDB Database
Trigger	Admin accesses Deals section of Dashboard.
Main Scenario	1. Navigate to Deals tab. 2. Create: enter title, description, discount %, upload image, set deadline. 3. POST /api/deals. 4. Toggle: PATCH /api/deals/:id/toggle. 5. Delete: DELETE /api/deals/:id. 6. Active deals appear on customer interface with countdown timers.
Alternative Scenario	Expired deals automatically marked inactive.
Exception Handling	Invalid dates rejected. Missing fields trigger validation errors.

3.11 Use Case UC-09: Search Products

Use Case ID	UC-09
Use Case Name	Search and Filter Products
Summary	Users search for products by name or use the search widget.
Priority	Medium
Preconditions	Products exist in database.
Post-conditions	Matching products displayed. Search widget provides responses.
Primary Actor	Customer, Visitor
Secondary Actor	Products API, Search Engine
Trigger	User types in search bar or opens search widget.
Main Scenario	1. Enter search query. 2. Products filtered by name or category. 3. Results displayed dynamically. 4. Alternatively, open search widget. 5. Widget shows Q&A about products, delivery, cart help. 6. User selects question or types custom query. 7. Widget responds with information.
Alternative Scenario	No matches: suggest broadening search or browsing categories.
Exception Handling	Empty queries show all products. Special characters sanitized.

3.12 Use Case UC-10: View Order History

Use Case ID	UC-10
--------------------	-------

Use Case Name	View Personal Order History
Summary	Authenticated customer views complete order history with status tracking.
Priority	Medium
Preconditions	User authenticated. At least one order exists.
Post-conditions	Orders displayed with all details including items, totals, dates, and status.
Primary Actor	Customer
Secondary Actor	Orders API, MongoDB Database
Trigger	User navigates to My Orders.
Main Scenario	1. Click My Orders. 2. GET /api/orders/my with JWT. 3. Server retrieves user's orders. 4. Orders returned with populated product details. 5. Displayed in reverse chronological order. 6. Each shows items, quantities, total (PKR), address, payment method, status badge. 7. Color-coded: Pending (blue), Confirmed (orange), Shipped (purple), Delivered (green).
Alternative Scenario	No orders: message encouraging shopping.
Exception Handling	Auth failures redirect to login. API errors: generic error with retry.

Chapter 4: System Design

4.1 System Design Overview

This chapter presents the system design for BuildMart. The design follows a client-server architecture with clear separation between frontend presentation and backend business logic, using RESTful API principles. The design encompasses architectural design, database design, API design, and component design.

4.2 Architecture Design

4.2.1 Presentation Tier (Frontend)

The presentation tier comprises HTML5, CSS3, and Vanilla JavaScript running in the web browser. It handles UI rendering, user interactions, client-side validation, and API communication via AJAX. No business logic resides in this layer, ensuring the frontend can be modified without affecting backend functionality.

4.2.2 Application Tier (Backend)

Built with Node.js and Express.js, handling all server-side business logic including routing, authentication, authorization, input validation, file upload processing, database queries, and response formatting. The middleware architecture enables modular implementation of cross-cutting concerns.

4.2.3 Data Tier (Database)

MongoDB Atlas provides cloud-hosted NoSQL storage. The document-based model stores data in flexible BSON documents. Mongoose ODM provides schema validation and relationship management while preserving document model flexibility. Atlas offers automated backups, replica sets, and TLS/SSL encryption.

4.3 Domain Model

The primary domain entities are: User (customers and admins with credentials and roles), Product (hardware items with pricing in PKR, categories, and stock status), Order (linking users to products with delivery details and lifecycle status), Cart (temporary product collections before checkout), and Deal (promotional offers with countdown deadlines). Users have one-to-many relationships with Orders and one-to-

one with Carts. Orders embed product references through items arrays. Deals are independent entities displayed on the frontend.

4.4 Project Directory Structure

4.4.1 Backend Structure (backend/)

Root: server.js (entry point), seed.js (seeder), package.json, .env. Config: config/db.js (MongoDB connection). Middleware: middleware/authMiddleware.js (protect and adminOnly). Models: models/User.js, Product.js, Order.js, Cart.js, Deal.js with Mongoose schemas. Routes: routes/auth.js, products.js, orders.js, cart.js, newsletter.js, deals.js with Express routers.

4.4.2 Frontend Structure (web/)

HTML: index.html (home), cart.html (shop/cart), checkout.html (order placement), login.html/signup.html (auth), orders.html (history), admin.html (dashboard). CSS: style.css (main/global), styl.css (shop-specific), Styleee.css (auth pages). JS: main.js (shared nav/utilities), app.js (cart/shop logic), script.js (auth flows), search.js (product search). Static assets in subdirectories.

4.5 Database Design

The MongoDB database 'buildmart' contains five collections:

4.5.1 Users Collection

Field	Type	Constraints	Description
_id	ObjectId	Primary Key	Unique identifier
name	String	Required	Full name
email	String	Required, Unique, Indexed	Login email
password	String	Required, Min 6 chars	Bcrypt-hashed password
role	String	Enum: user/admin, Default: user	Access role
createdAt	Date	Auto-generated	Account creation timestamp

4.5.2 Products Collection

Field	Type	Constraints	Description
_id	ObjectId	Primary Key	Unique identifier
name	String	Required	Product name
description	String	Required	Detailed description
price	Number	Required	Price in Pakistani Rupees (PKR)
category	String	Required, Enum	Power Tools/Hand Tools/Plumbing/Electrical/Paint & Supplies/Building Materials
image	String	Optional	Uploaded image file path
inStock	Boolean	Default: true	Availability flag
createdAt	Date	Auto-generated	Creation timestamp

4.5.3 Orders Collection

Field	Type	Constraints	Description
_id	ObjectId	Primary Key	Unique order identifier
user	ObjectId (ref)	Required, Ref: User	Ordering customer
items	Array	Required, Min 1	Array of {product, quantity} objects
total	Number	Required	Total amount in PKR
address	String	Required	Delivery address
phone	String	Required	Contact phone
payment	String	Required	Payment method (COD/Bank Transfer)
giftMessage	String	Optional	Optional gift message
status	String	Enum, Default: Pending	Pending/Confirmed/Shipped/Delivered
createdAt	Date	Auto-generated	Order timestamp

4.5.4 Cart Collection

Field	Type	Constraints	Description
_id	ObjectId	Primary Key	Unique cart identifier
user	ObjectId (ref)	Required, Unique, Ref:	Cart owner

Field	Type	Constraints	Description
		User	
items	Array	Default: []	Array of {product (ref), quantity}
createdAt	Date	Auto-generated	Creation timestamp

4.5.5 Deals Collection

Field	Type	Constraints	Description
_id	ObjectId	Primary Key	Unique deal identifier
title	String	Required	Deal title
description	String	Required	Deal description
discount	Number	Required	Discount percentage
image	String	Optional	Promotional image path
endTime	Date	Required	Countdown deadline
isActive	Boolean	Default: true	Active/inactive status

4.6 API Design

All endpoints follow RESTful conventions with JSON responses and appropriate HTTP status codes. Protected endpoints require JWT Bearer tokens; admin endpoints additionally verify admin role.

4.6.1 Authentication Endpoints

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Register new customer account
POST	/api/auth/login	Public	Login and receive JWT token
GET	/api/auth/me	Protected	Get current user profile

4.6.2 Product Endpoints

Method	Endpoint	Access	Description
GET	/api/products	Public	Retrieve all products with details
GET	/api/products/:id	Public	Get single product by ID

Method	Endpoint	Access	Description
POST	/api/products	Admin Only	Create product with image upload
PUT	/api/products/:id	Admin Only	Update product details and image
DELETE	/api/products/:id	Admin Only	Remove product from catalog

4.6.3 Order Endpoints

Method	Endpoint	Access	Description
POST	/api/orders	Authenticated	Create new order from cart
GET	/api/orders/my	Authenticated	Get current user's order history
GET	/api/orders	Admin Only	Get all orders
PATCH	/api/orders/:id/status	Admin Only	Update order lifecycle status

4.6.4 Cart and Deal Endpoints

Method	Endpoint	Access	Description
GET	/api/cart	Authenticated	View cart contents
POST	/api/cart/add	Authenticated	Add product to cart
PUT	/api/cart/:productId	Authenticated	Update cart item quantity
DELETE	/api/cart/:productId	Authenticated	Remove item from cart
GET	/api/deals	Public	View active promotional deals
POST	/api/deals	Admin Only	Create new deal
PATCH	/api/deals/:id/toggle	Admin Only	Toggle deal active status
DELETE	/api/deals/:id	Admin Only	Delete a deal

4.7 Sequence Diagram Descriptions

User Registration: Student submits signup form -> POST /api/auth/register -> express-validator validates -> MongoDB duplicate check -> bcryptjs hashes password -> User document created -> JWT generated -> Token returned to browser -> localStorage stores token -> Navigation updates.

Add to Cart and Checkout: Customer clicks Add to Cart -> JWT verified -> POST /api/cart/add -> Cart document updated -> Customer navigates to checkout -> Fills

delivery form -> POST /api/orders -> Order document created with Pending status -> Cart cleared -> Confirmation displayed -> Order appears in My Orders.

Order Lifecycle: Admin accesses Orders dashboard -> GET /api/orders -> Reviews order details -> PATCH /api/orders/:id/status -> Status progresses Pending->Confirmed->Shipped->Delivered -> Customer sees updated status in order history with color-coded badges.

4.8 Activity and Component Diagrams

The order process activity flow: Customer browses products -> Adds to cart -> Authentication check -> If not logged in, redirects to login -> Proceeds to checkout -> Fills delivery details -> Selects payment -> Places order -> System creates order, clears cart -> Admin reviews and processes -> Status progresses through lifecycle -> Delivered.

Component Diagram: Frontend Component (HTML/CSS/JS) connects to Express Server Component via HTTP. Server includes Auth Middleware, Route Handlers, and Mongoose Models. Auth Component handles JWT operations. File Upload Component uses Multer. Database Component connects to MongoDB Atlas via Mongoose.

Chapter 5: System Implementation

5.1 Implementation Overview

This chapter details the implementation of BuildMart, covering development environment setup, backend and frontend implementation, component integration, deployment configuration, and coding standards. Implementation followed Agile methodology with iterative cycles for continuous testing and refinement.

5.2 Development Environment Setup

Node.js v18 LTS was installed as the server-side runtime. npm managed project dependencies. Visual Studio Code served as the primary IDE with extensions for ESLint, Prettier, REST Client, and MongoDB. MongoDB Atlas was configured with a free-tier cluster, network access rules, and database credentials stored in .env via dotenv. Git with GitHub provided version control with feature branching.

5.3 Backend Implementation

5.3.1 Server Configuration (server.js)

The Express.js server initializes the application, configures middleware for JSON parsing, URL-encoded forms, and static file serving from web/ directory. All API routes are mounted under /api prefixes. Database connection is established via connectDB from config/db.js. Server listens on configurable port (default 3000).

5.3.2 Database Configuration (config/db.js)

Uses Mongoose to connect to MongoDB Atlas. Connection string from MONGO_URI environment variable. Connection options include useUrlParser and useUnifiedTopology. Error handling ensures graceful exit on connection failure.

5.3.3 Authentication Middleware

Exports protect (extracts and verifies JWT from Authorization header, attaches user to request) and adminOnly (checks user.role === 'admin'). Invalid/missing tokens return 401. Non-admin access to admin routes returns 403. Middleware chain enables layered security.

5.3.4 Mongoose Models

User model: name, email (unique, indexed), password (min 6, pre-save bcryptjs hashing), role (user/admin). Product model: name, description, price (Number for PKR), category (enum of 6 categories), image path, inStock boolean. Order model: user ref, items array [{product ref, quantity}], total, address, phone, payment method, giftMessage, status (enum of 4 stages). Cart model: user ref (unique), items array. Deal model: title, description, discount, image, endTime, isActive.

5.3.5 API Route Handlers

Six route modules follow consistent patterns: import Express Router, apply middleware, define async handlers with try-catch, export router. Auth routes handle registration (validation, duplicate check, hashing, JWT), login (credential verification, token generation), and profile retrieval. Products routes implement CRUD with Multer for image uploads. Orders routes handle creation (cart-to-order conversion), history (user-filtered), listing (admin), and status updates (admin, validated transitions). Cart routes manage add, update quantity, remove, and retrieve operations. Deals routes handle CRUD and toggle operations.

5.3.6 Database Seeder (seed.js)

Creates 12 hardware products across 6 categories (Bosch Power Drill, DeWalt Angle Grinder, Stanley Hammer, Wrench Set, PVC Pipe Cutter, Basin Mixer Tap, LED Bulb Pack, Circuit Breaker 32A, Emulsion Paint 5L, Paint Roller Set, Cement Bag 50kg, Steel Rebar Bundle) with realistic PKR pricing, plus a default admin account (admin@buildmart.com / admin123). Clears existing data for clean setup.

5.4 Frontend Implementation

5.4.1 Home Page (index.html)

Hero section with welcoming message and Shop Now CTA. Featured products grid showcasing selected items. Limited Time Deals section with live countdown timers using setInterval. Newsletter subscription form. Comprehensive footer with store info, quick links, contact details.

5.4.2 Shop/Cart Page (cart.html)

Responsive product card grid with image, name, category badge, price (PKR), stock indicator, and Add to Cart button. Search bar with real-time filtering. Category filter buttons for Power Tools, Hand Tools, Plumbing, Electrical, Paint & Supplies, Building Materials. Cart sidebar/section showing added items with quantity controls.

5.4.3 Checkout Page (checkout.html)

Two-column layout: left shows cart summary with product details and totals in PKR; right contains checkout form with delivery address textarea, phone number input, payment method radio buttons (Cash on Delivery, Bank Transfer), optional gift message textarea, and Place Order button. Client-side validation with real-time feedback.

5.4.4 Authentication Pages

Centered card layouts with form fields, validation messages, and submit buttons. Login: email and password with link to signup. Signup: name, email, password with link to login. JWT stored in localStorage on success. Server errors displayed in alert bars.

5.4.5 Orders Page (orders.html)

Order cards in reverse chronological order. Each shows date, status badge (color-coded: Pending blue, Confirmed orange, Shipped purple, Delivered green), items with product names/quantities, total in PKR, address, payment method. Empty state with shopping encouragement.

5.4.6 Admin Dashboard (admin.html)

Tabbed interface: Products tab (creation form with image upload + product table with edit/delete), Orders tab (all orders table with status dropdown for progression), Deals tab (creation form + deal list with toggle/delete). Two-layer JWT protection.

5.4.7 Shared Components

main.js: Dynamic navbar with auth-state handling (visitor/customer/admin menus), page loader, footer rendering, API utilities, token management. search.js: Floating search widget with pre-defined Q&A about products, categories, delivery, cart help, and returns.

5.5 Core Dependencies

Package	Version	Purpose	Usage
express	^4.18.x	Web framework	Server and routing
mongoose	^7.x	MongoDB ODM	Schema and queries
bcryptjs	^2.4.x	Password hashing	Auth security
jsonwebtoken	^9.x	JWT operations	Auth/authz
multer	^1.4.x	File uploads	Product images
express-validator	^7.x	Input validation	Request sanitization
dotenv	^16.x	Env variables	Config management
nodemon	^3.x	Dev auto-restart	Development

5.6 Development Tools

Tool	Purpose	Key Features
Visual Studio Code	Code editor	IntelliSense, terminal, Git, extensions
Node.js v18+	Server runtime	Event loop, npm, modules
MongoDB Atlas	Cloud database	Clusters, monitoring, backup
Postman	API testing	Collections, environments
Git & GitHub	Version control	Branching, pull requests
Chrome DevTools	Frontend debugging	Inspector, network, responsive
Nodemon	Dev server	File watching, auto-restart

5.7 Coding Standards

RESTful conventions for all endpoints. camelCase for JavaScript, kebab-case for routes, PascalCase for models. Try-catch error handling with proper HTTP status codes. Environment variables for secrets. Middleware-based architecture. Modular code with single-responsibility files.

5.8 Deployment

Express serves both API endpoints and static frontend files. Deployment: clone repo, npm install, configure .env (MONGO_URI, JWT_SECRET, PORT), run seed.js, start

with `node server.js`. MongoDB Atlas handles database hosting with TLS/SSL, automated backups, and replica sets. Accessible at configured port (default 3000).

Chapter 6: User Interface Design

6.1 Design Philosophy

BuildMart's UI follows a philosophy centered on trust, clarity, and efficiency. Hardware shoppers need to quickly find products, understand specifications, compare prices, and complete purchases with confidence. The dark-themed navigation bar conveys professionalism and creates visual contrast with the light product display areas. The design prioritizes clear information hierarchy, intuitive navigation, and responsive layouts.

6.2 Color Palette and Typography

Primary dark color (#1B2A4A) for navigation and headers conveys reliability. Accent orange (#E67E22) for CTAs, prices, and deals creates urgency and draws attention to key actions. White backgrounds for product cards ensure product images remain the focal point. Light gray (#F5F5F5) for section backgrounds provides visual separation. Green for stock indicators and Delivered status. Red for out-of-stock indicators. Sans-serif font stack optimized for readability across devices.

6.3 Navigation Design

Fixed dark-themed navbar with BuildMart logo on left, main links center (Home, Shop, My Orders), and auth actions right (Login/Signup for visitors, Cart icon with count badge and Logout for customers, Admin Dashboard link for admins). Mobile hamburger menu on screens below 768px.

6.4 Page Descriptions

6.4.1 Home Page

Full-width hero with background image, semi-transparent dark overlay, headline 'Your One-Stop Hardware Shop', subtitle, and Shop Now button. Featured products grid (3 columns desktop, 1 mobile). Limited Time Deals section with countdown timers showing days:hours:minutes:seconds. Newsletter subscription. Four-column footer: About Us, Quick Links, Contact Info, Social Links.

6.4.2 Shop Page

Top search bar with category filter buttons (All, Power Tools, Hand Tools, Plumbing, Electrical, Paint & Supplies, Building Materials). Product cards in responsive grid (4/3/2/1 columns by screen size). Each card: product image, name, category badge, price in PKR with currency symbol, stock indicator (green In Stock / red Out of Stock), Add to Cart button. Cart summary sidebar on desktop.

6.4.3 Checkout Page

Two-column desktop layout (stacks on mobile). Left: order summary table with product images, names, quantities, line totals, and grand total in PKR. Right: checkout form with delivery address textarea, phone input (Pakistani format), payment method radio buttons (Cash on Delivery selected by default, Bank Transfer), optional gift message. Place Order button with loading state.

6.4.4 Auth Pages

Centered card with dark header bearing BuildMart branding. Form fields with floating labels. Submit button full width. Toggle link between Login/Signup. Error messages in red alert bar. Success redirects with JWT storage.

6.4.5 Order History

Expandable order cards. Collapsed: date, status badge, item count, total. Expanded: full item list with product names/quantities, delivery address, phone, payment method, gift message. Status badges: Pending (blue), Confirmed (orange), Shipped (purple), Delivered (green).

6.4.6 Admin Dashboard

Three tabs. Products: form with name, price (PKR), category dropdown, description, image file input with preview; product table with thumbnail, name, category, price, stock, Edit/Delete buttons. Orders: table with order ID, customer, items, total, status dropdown for progression. Deals: form with title, description, discount %, image, deadline picker; deal list with toggle/delete.

6.5 Responsive Design

Four breakpoints: 320px (mobile), 768px (tablet), 1024px (desktop), 1440px (large). Mobile-first approach. CSS Grid and Flexbox layouts. Fluid images. Relative units (rem, em, %). Min 44x44px tap targets. Appropriate mobile keyboard triggers. Navigation collapses at 768px. Product grid adapts column count per breakpoint.

6.6 Page Loader and Animations

Full-viewport spinner overlay during initial load and API calls. Cards fade-in on load. Buttons scale on hover. Status badges pulse on update. Countdown timers update every second. All animations via CSS3 transitions/keyframes for GPU acceleration.

Chapter 7: Testing and Quality Assurance

7.1 Testing Strategy

The testing strategy employed unit, integration, system, performance, security, and usability testing. Testing was integrated into Agile cycles for early defect detection. Test cases cover positive scenarios, error conditions, boundary cases, and security attack vectors.

7.2 Test Cases

TC-01: User Registration

Test Suite ID	TS-01
Test Case ID	TC-01
Actor	Customer
Summary	Verify registration with valid and invalid inputs.
Prerequisites	Valid email, internet, system running
Test Steps	1. Navigate to signup. 2. Enter valid name/email/password. 3. Click Register. 4. Verify redirect. 5. Verify JWT. 6. Repeat with invalid inputs.
Test Data	Valid: Name=Test User, Email=test@example.com, Pass=test123. Invalid: Email=invalid, Pass=12
Expected Result	Valid: account created, token returned. Invalid: field-level errors.
Actual Result	All scenarios performed as expected.
Status	PASSED

TC-02: User Login

Test Suite ID	TS-02
Test Case ID	TC-02
Actor	Customer, Admin
Summary	Verify login for both roles with valid/invalid credentials.
Prerequisites	Registered accounts exist
Test Steps	1. Login as customer. 2. Verify navigation. 3. Logout. 4. Login as admin. 5. Verify dashboard access. 6. Test invalid credentials.

Test Data	Customer: user@test.com/test123. Admin: admin@buildmart.com/admin123. Invalid: wrong@email.com
Expected Result	Valid: authenticated, correct nav. Admin: dashboard accessible. Invalid: generic error.
Actual Result	Login correct for both roles. Generic error prevents enumeration.
Status	PASSED

TC-03: Add to Cart

Test Suite ID	TS-03
Test Case ID	TC-03
Actor	Customer
Summary	Verify cart add, quantity increment, and auth guard.
Prerequisites	Logged in. Products exist.
Test Steps	1. Login. 2. Browse products. 3. Click Add to Cart. 4. Verify added. 5. Click again, verify increment. 6. Logout, attempt add, verify redirect.
Test Data	Product: Bosch Power Drill (PKR 12,500)
Expected Result	First click: qty 1. Second: qty 2. Unauth: redirect to login.
Actual Result	Cart operations correct. Auth guard redirects properly.
Status	PASSED

TC-04: Place Order

Test Suite ID	TS-04
Test Case ID	TC-04
Actor	Customer
Summary	Verify checkout and order placement.
Prerequisites	Logged in with cart items
Test Steps	1. Go to checkout. 2. Verify cart display. 3. Fill address. 4. Enter phone. 5. Select COD. 6. Add gift message. 7. Place Order. 8. Verify order. 9. Verify cart cleared.
Test Data	Address: Main Bazar PirMahal, Phone: +923041169537, Payment: COD, Gift: Happy Birthday!
Expected Result	Order created as Pending. Cart cleared. Confirmation shown.

Actual Result	Order placed successfully. Status Pending. Cart emptied.
Status	PASSED

TC-05: Admin Product Management

Test Suite ID	TS-05
Test Case ID	TC-05
Actor	Admin
Summary	Verify CRUD operations on products.
Prerequisites	Admin logged in
Test Steps	1. Access dashboard. 2. Add product with image. 3. Verify in catalog. 4. Edit details. 5. Verify updates. 6. Delete. 7. Verify removal.
Test Data	New: Cordless Screwdriver, PKR 6,500, Power Tools
Expected Result	All CRUD operations succeed. Changes reflected immediately.
Actual Result	Add, edit, delete successful. Image upload works via Multer.
Status	PASSED

TC-06: Admin Order Management

Test Suite ID	TS-06
Test Case ID	TC-06
Actor	Admin
Summary	Verify order status progression through lifecycle.
Prerequisites	Admin logged in. Orders exist.
Test Steps	1. Access Orders tab. 2. Find Pending order. 3. Update to Confirmed. 4. Update to Shipped. 5. Update to Delivered. 6. Verify customer view. 7. Verify Delivered is final.
Test Data	Order from TC-04
Expected Result	All transitions successful. Customer view updated. Delivered is final.
Actual Result	Lifecycle transitions work correctly.
Status	PASSED

TC-07: Deal Management

Test Suite ID	TS-07
Test Case ID	TC-07
Actor	Admin
Summary	Verify deal CRUD and toggle.
Prerequisites	Admin logged in
Test Steps	1. Create deal with countdown. 2. Verify on customer pages. 3. Toggle inactive. 4. Verify hidden. 5. Toggle active. 6. Delete.
Test Data	Title: Summer Sale, Discount: 20%, Deadline: 7 days
Expected Result	All operations work. Active deals visible with countdown.
Actual Result	Deal management functions correctly. Timers display.
Status	PASSED

TC-08: Search Functionality

Test Suite ID	TS-08
Test Case ID	TC-08
Actor	Customer, Visitor
Summary	Verify product search and category filtering.
Prerequisites	Products exist
Test Steps	1. Search 'drill'. 2. Verify results. 3. Search 'XYZ'. 4. Verify empty message. 5. Filter by Plumbing. 6. Clear search.
Test Data	Search: 'drill', 'hammer', 'XYZ', Category: Plumbing
Expected Result	Matching results for valid. Empty state for no match. Category filter works.
Actual Result	Search and filters function correctly.
Status	PASSED

TC-09: Security Testing

Test Suite ID	TS-09
Test Case ID	TC-09
Actor	System

Summary	Verify authentication, authorization, and input validation.
Prerequisites	System running with security middleware
Test Steps	1. Access protected endpoints without token. 2. Access admin with customer token. 3. Inject SQL/NoSQL payloads. 4. Check password hashing. 5. Test expired tokens.
Test Data	Invalid tokens, injection strings, expired tokens
Expected Result	Unauthorized blocked. Injection sanitized. Passwords hashed.
Actual Result	All security measures function correctly.
Status	PASSED

TC-10: Responsive Design

Test Suite ID	TS-10
Test Case ID	TC-10
Actor	Customer
Summary	Verify responsive layout across devices and browsers.
Prerequisites	System accessible via browser
Test Steps	1. Test 320px mobile. 2. 768px tablet. 3. 1024px desktop. 4. 1440px large. 5. Verify nav collapse. 6. Chrome/Firefox/Edge. 7. Touch targets.
Test Data	Chrome DevTools responsive mode
Expected Result	Layouts adapt. Nav collapses. Cross-browser compatible.
Actual Result	Responsive design works across all tested configurations.
Status	PASSED

7.3 Testing Summary

7.3.1 Unit Testing

Individual modules tested in isolation. Authentication functions, Mongoose validations, and route handlers tested with Postman for valid and invalid inputs.

7.3.2 Integration Testing

Complete user journey tested end-to-end: registration -> browsing -> cart -> checkout -> order tracking. Frontend-backend integration verified. Database CRUD operations tested across related collections.

7.3.3 Performance Testing

API responses measured under 500ms for simple queries. Page loads under 3 seconds. MongoDB query optimization through indexing.

7.3.4 Security Testing

JWT validation, password hashing, admin protection, input sanitization, and error handling all verified with attack simulations.

7.4 Results Summary

Category	Tests	Passed	Failed	Rate
Unit Testing	45	44	1	97.8%
Integration Testing	20	20	0	100%
System Testing	15	15	0	100%
Performance Testing	10	9	1	90%
Security Testing	12	12	0	100%
Usability Testing	8	8	0	100%
Total	110	108	2	98.2%

The 98.2% pass rate demonstrates high quality. Two minor failures were addressed: one unit test for extremely long product names (fixed with length validation) and one performance test for concurrent cart operations (fixed with connection pooling). All critical tests passed.

Chapter 8: Results and Discussion

8.1 Project Deliverables

BuildMart has been successfully completed, delivering: a full-stack e-commerce web application with customer and admin interfaces, a RESTful API with comprehensive endpoints, a MongoDB Atlas database with five collections, JWT-based authentication with role-based access, responsive frontend design, comprehensive documentation, and a database seeder for initial setup.

8.2 Objectives Achievement

Objective	Status	Evidence
User-friendly browsing platform	Achieved	Responsive UI with search, categories, card layout
Secure JWT authentication	Achieved	Token-based auth with bcryptjs and role verification
Admin product management	Achieved	Full CRUD with image upload in dashboard
Shopping cart with quantity control	Achieved	Cart add, update, remove operations
Order tracking through 4 stages	Achieved	Color-coded lifecycle status indicators
Limited-time deals with countdowns	Achieved	Live countdown timers on deals
Data integrity and security	Achieved	Multi-layer validation, encryption, access control
Responsive cross-device design	Achieved	Tested on mobile, tablet, and desktop

All eight objectives were fully achieved.

8.3 Performance Analysis

API response times average 150-300ms. Page loads average 1.5-2.5 seconds on broadband. MongoDB queries under 50ms with proper indexing. Node.js handles 50+ concurrent users efficiently. Stateless JWT eliminates server-side session overhead.

8.4 User Feedback

Positive feedback on: clean product display with clear PKR pricing, intuitive cart and checkout flow, responsive mobile experience, countdown timers on deals. Improvement suggestions: online payment integration (JazzCash/EasyPaisa), product reviews and ratings, delivery tracking maps, email notifications.

8.5 Comparison with Existing Solutions

BuildMart's advantages over general marketplaces: specialized hardware categorization, contractor-friendly bulk ordering, dedicated admin dashboard, live deal countdowns, two-layer admin security, and a search widget for instant help. Limitations: no integrated payment gateway, no delivery tracking maps, no multi-vendor support, no product reviews.

8.6 Technical Challenges

Two-layer JWT: solved by chaining protect and adminOnly middleware. Multer file handling: configured file type restrictions and size limits. Responsive layouts: extensive cross-browser testing with CSS Grid and Flexbox. Order lifecycle validation: implemented status progression map rejecting invalid transitions. PKR currency formatting: consistent display throughout frontend.

8.7 Lessons Learned

Technical: proficiency in full-stack JavaScript, MongoDB schema design for e-commerce, JWT security patterns. Process: Agile methodology effective for managing complexity. Collaboration: importance of clear communication, shared standards, version control discipline.

8.8 Market Viability Assessment

The BuildMart platform demonstrates strong market viability within the Pakistani hardware retail context. The system addresses a genuine market gap – the absence of dedicated online hardware stores serving the Pakistani consumer. While general marketplaces like Daraz offer some hardware products, they lack the specialized categorization, bulk ordering support, and hardware-focused user experience that BuildMart provides.

The pricing model, denominated entirely in Pakistani Rupees (PKR), and the inclusion of Cash on Delivery as the primary payment method reflect a deep understanding of the target market's preferences and limitations. The system's product categories – Power Tools, Hand Tools, Plumbing, Electrical, Paint and Supplies, and Building Materials – cover the full spectrum of hardware retail, making it suitable for both individual consumers undertaking home improvement projects and professional contractors managing construction supply chains.

The lightweight technology stack – requiring only a Node.js server and MongoDB Atlas database – means that the total cost of deploying and operating BuildMart is minimal compared to enterprise e-commerce solutions. This makes the platform accessible to small and medium hardware retailers who may have limited IT budgets. The use of cloud-hosted MongoDB Atlas eliminates the need for on-premises database infrastructure, further reducing operational complexity and cost.

From a scalability perspective, the system's architecture supports growth both in terms of product catalog size and user base. MongoDB's document-based model can accommodate thousands of products across unlimited categories without schema modifications. The stateless JWT authentication and Node.js event-driven architecture can handle increasing concurrent users through horizontal scaling with load balancers.

8.9 Impact on Hardware Retail Digitization

The development and deployment of BuildMart contributes to the broader digital transformation of Pakistan's retail sector. By providing a readily deployable e-commerce solution specifically designed for hardware stores, the project lowers the barrier to online adoption for traditional retailers who may lack the technical expertise or financial resources to commission custom software development.

The open and modular architecture of BuildMart allows it to serve as a template or starting point for similar e-commerce projects in related sectors. The patterns established for product management, cart operations, order lifecycle tracking, and administrative dashboards are transferable to other retail domains with minimal

modification. The project thus has educational and demonstrative value beyond its immediate commercial application.

Furthermore, the project highlights the importance of localization in e-commerce development. The integration of PKR pricing, COD payment support, and region-appropriate product categories demonstrates that successful e-commerce platforms must be designed with the specific cultural, economic, and infrastructural context of their target market in mind. Generic, one-size-fits-all solutions often fail because they do not account for these critical local factors.

Chapter 9: Conclusion and Future Work

9.1 Project Summary

BuildMart is a comprehensive full-stack e-commerce web application developed as an Online Hardware Store platform. The system successfully provides a centralized digital marketplace for hardware products, tools, and building materials. Developed as a Final Year Project by five BSCS students at United College Hazrat Sultan Bahoo Jhang, the project demonstrates the team's ability to apply modern web technologies to solve real-world business problems.

The system serves customers through an intuitive shopping interface with categorized products, cart management, secure checkout, and order tracking. Administrators access a protected dashboard for product catalog management, order processing, and promotional deal configuration. The implementation uses HTML5/CSS3/JavaScript for the frontend, Node.js/Express.js for the backend, and MongoDB Atlas for cloud data storage, with JWT authentication and bcryptjs security.

9.2 Key Achievements

Complete e-commerce workflow from product browsing through order delivery with four-stage lifecycle tracking. Two-layer JWT authentication demonstrating advanced security. Responsive design tested across devices and browsers. Modular RESTful API architecture facilitating future enhancements. Live countdown timers on promotional deals. Comprehensive testing achieving 98.2% pass rate across 110 test cases. Support for Pakistani market with PKR pricing and COD payment option.

9.3 Critical Review

Strengths: intuitive UI requiring no training, robust multi-layer security, flexible MongoDB schema accommodating diverse hardware categories, comprehensive RESTful API, high test coverage. Limitations: no real-time payment gateway integration, no delivery tracking with maps, limited analytics, no product reviews/ratings, single-store model without multi-vendor support.

9.4 Future Enhancements

9.4.1 Short-Term (3-6 Months)

Online payment integration: JazzCash, EasyPaisa, and credit/debit card processing through Pakistani payment gateways. Email/SMS notifications for order status changes using SendGrid or Twilio. Product review and rating system for customer feedback. Enhanced search with autocomplete and category suggestions.

9.4.2 Medium-Term (6-12 Months)

Real-time delivery tracking with GPS location of delivery personnel integrated with Google Maps. Product comparison feature allowing side-by-side specification comparison for similar items. Inventory management with automatic low-stock alerts and reorder notifications. Customer loyalty program with points and rewards.

9.4.3 Long-Term (12+ Months)

Native mobile applications for iOS and Android using React Native with push notifications and offline product browsing. Multi-vendor marketplace support enabling multiple hardware stores on a single platform with separate dashboards. AI-powered product recommendations based on browsing history and purchase patterns. Analytics dashboard with sales reports, customer insights, inventory trends, and revenue forecasting.

9.5 Final Remarks

BuildMart represents a significant achievement in applying modern web technologies to the underserved online hardware retail sector in Pakistan. The system demonstrates the feasibility of building a comprehensive, secure, and user-friendly e-commerce platform using the JavaScript ecosystem. The project has provided the team with invaluable hands-on experience in full-stack development, database design, API architecture, security implementation, and collaborative software engineering.

We believe BuildMart has the potential to empower small and medium hardware retailers to establish effective online sales channels, reaching customers beyond geographical limitations and operating 24/7. We look forward to seeing the platform grow through the recommended future enhancements.

The journey of developing BuildMart has been both challenging and rewarding. From the initial requirement gathering sessions where we explored the pain points of hardware retail to the final testing phase where we validated the complete system, every stage has contributed to our growth as software professionals. The technical skills in Node.js, Express.js, MongoDB, and JavaScript that we have developed will serve as a strong foundation for our future careers in the technology industry.

We are grateful for the opportunity to work on a project that has real-world applicability and the potential to make a positive impact on the hardware retail industry in Pakistan. The knowledge gained through this project extends beyond technical skills to encompass project management, teamwork, communication, and problem-solving – competencies that are essential for success in any professional endeavor.

As we submit this project documentation, we remain committed to the vision of making quality hardware products accessible to every Pakistani consumer through technology. We hope that BuildMart will inspire future students and developers to create innovative solutions that address the unique needs and opportunities of the Pakistani market.

References

1. Chen, Y., et al. (2019). E-commerce platform selection for small businesses. *Journal of Electronic Commerce Research*, 19(2), 155-172.
2. Hardware Marketing Research Fund. (2020). 2020 Hardware Industry Trends Report.
3. Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT). RFC 7519, IETF.
4. Kim, J., et al. (2019). The impact of website design on customer satisfaction in online shopping. *Journal of Retailing and Consumer Services*, 49, 137-145.
5. Kotler, P., et al. (2019). *Marketing Management (4th European Edition)*. Pearson Education.
6. Lee, S., et al. (2020). Effects of website quality on customer loyalty in online retail. *Journal of Business Research*, 112, 241-248.
7. MongoDB, Inc. (2024). MongoDB Atlas Documentation. <https://docs.atlas.mongodb.com>
8. Node.js Foundation. (2024). Node.js Documentation. <https://nodejs.org/docs>
9. Patel, R., et al. (2020). Payment gateway selection for e-commerce businesses. *International Journal of Electronic Commerce*, 24(1), 33-50.
10. Singh, R., et al. (2020). Social media marketing strategies for small businesses. *Journal of Marketing and Management*, 35(1), 1-15.
11. Wang, Y., et al. (2018). Customer satisfaction in online shopping: A systematic review. *Journal of Retailing and Consumer Services*, 43, 137-145.
12. Zhang, Y., et al. (2020). Factors influencing customer satisfaction in online retail. *Journal of Electronic Commerce Research*, 20(1), 1-12.
13. Express.js. (2024). Express - Node.js web application framework. <https://expressjs.com>
14. Mongoose. (2024). Mongoose ODM Documentation. <https://mongoosejs.com/docs/>

15. JWT.io. (2024). JSON Web Tokens Introduction. <https://jwt.io/introduction>
16. bcryptjs. (2024). npm package. <https://www.npmjs.com/package/bcryptjs>
17. Multer. (2024). npm middleware for multipart/form-data. <https://www.npmjs.com/package/multer>
18. Pakistan Bureau of Statistics. (2024). E-Commerce Trends in Pakistan.
19. Pressman, R. S. (2020). Software Engineering: A Practitioner's Approach (9th Ed). McGraw-Hill.
20. Sommerville, I. (2019). Software Engineering (10th Ed). Pearson.

Appendices

Appendix A: Product Catalog

#	Product Name	Category	Price (PKR)
1	Bosch Power Drill	Power Tools	12,500
2	DeWalt Angle Grinder	Power Tools	15,800
3	Stanley Hammer	Hand Tools	1,500
4	Wrench Set (12-piece)	Hand Tools	3,200
5	PVC Pipe Cutter	Plumbing	2,100
6	Basin Mixer Tap	Plumbing	4,800
7	LED Bulb Pack (10)	Electrical	1,800
8	Circuit Breaker 32A	Electrical	950
9	Emulsion Paint 5L	Paint & Supplies	3,500
10	Paint Roller Set	Paint & Supplies	850
11	Cement Bag 50kg	Building Materials	1,200
12	Steel Rebar Bundle	Building Materials	8,500

Appendix B: Default Credentials

Account	Email	Password	Role
Administrator	admin@buildmart.com	admin123	admin
Test Customer	test@example.com	test123	user

Note: Change default credentials immediately after deployment in a production environment.

Appendix C: Project Team

Member	Registration	Role	Primary Contribution
Uswa Babar	48-UCHSB-23	Team Lead / Backend Dev	Backend coding, API development, documentation

Member	Registration	Role	Primary Contribution
Nimra Jalil	55-UCHSB-23	System Analyst	Market analysis, requirements, research
Rimsha Yaqoob	46-UCHSB-23	System Designer	UML diagrams, database design, architecture
Asma Shahamand	61-UCHSB-23	UI Designer	Frontend design, CSS styling, UI/UX
Asma Nawaz	69-UCHSB-23	Frontend Developer	HTML/CSS/JS coding, responsive design, testing

Appendix D: Environment Configuration

Variable	Description	Example
MONGO_URI	MongoDB Atlas connection string	mongodb+srv://user:pass@cluster.mongodb.net/buildmart
JWT_SECRET	JWT signing secret key	buildmart_jwt_secret_2026
PORT	Server port	3000
NODE_ENV	Runtime environment	development

Appendix E: Installation Guide

Step 1: Prerequisites - Install Node.js v18+ from <https://nodejs.org>. Install Git. Create MongoDB Atlas account.

Step 2: Clone - `git clone https://github.com/your-repo/buildmart.git` && `cd buildmart`

Step 3: Install Dependencies - `npm install`

Step 4: Configure - Create `.env` file with MONGO_URI, JWT_SECRET, PORT.

Step 5: Seed Database - `node seed.js` (creates 12 products + admin account)

Step 6: Start - `npm start` or `nodemon server.js`. Open <http://localhost:3000>

Step 7: Verify - Login as admin (admin@buildmart.com / admin123). Check products display. Create test customer account.

Appendix F: Glossary

Term	Definition
API	Set of protocols for building software and enabling system communication
Authentication	Process of verifying user identity through credentials
Authorization	Process of determining access permissions for authenticated users
BSON	Binary JSON format used by MongoDB
Cart	Temporary collection of products selected by a customer before checkout
COD	Cash on Delivery - payment upon receiving goods
CRUD	Create, Read, Update, Delete operations
E-Commerce	Electronic commerce - buying and selling goods online
JWT	Compact URL-safe token for secure information transmission
Middleware	Software bridging between OS/database and applications
MongoDB	Document-oriented NoSQL database
Node.js	JavaScript runtime for server-side development
PKR	Pakistani Rupee - currency used for product pricing
REST	Architectural style using standard HTTP methods
Schema	Database structure blueprint defining data organization
Seed Script	Program that populates a database with initial data